

In [1]:

```
#import relevant modules
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
```

In [2]:

```
#bring csv into the the workbook
data = pd.read_csv('D:/AI Course/Applied Machine Intelligence/covid-vaccination-vs-death_ra
```

In [3]:

```
data.columns
```

Out[3]:

```
Index(['Unnamed: 0', 'country', 'iso_code', 'date', 'total_vaccinations',
      'people_vaccinated', 'people_fully_vaccinated', 'New_deaths',
      'population', 'ratio'],
      dtype='object')
```

In [4]:

```
#understand the dataframe
data
```

Out[4]:

	Unnamed: 0	country	iso_code	date	total_vaccinations	people_vaccinated	people_f
0	0	Afghanistan	AFG	2021-05-11	504502.0	448878.0	
1	1	Afghanistan	AFG	2021-05-20	547901.0	470341.0	
2	2	Afghanistan	AFG	2021-05-24	573277.0	476367.0	
3	3	Afghanistan	AFG	2021-05-26	590454.0	479372.0	
4	4	Afghanistan	AFG	2021-05-27	593313.0	479574.0	
...	...	...	...	...	...	...	
32906	32906	Zimbabwe	ZWE	2022-03-25	8691642.0	4814582.0	
32907	32907	Zimbabwe	ZWE	2022-03-26	8791728.0	4886242.0	
32908	32908	Zimbabwe	ZWE	2022-03-27	8845039.0	4918147.0	
32909	32909	Zimbabwe	ZWE	2022-03-28	8934360.0	4975433.0	
32910	32910	Zimbabwe	ZWE	2022-03-29	9039729.0	5053114.0	

32911 rows × 10 columns



In [5]:

```
#remove unnecessary columns
data.drop(columns=['Unnamed: 0', 'iso_code', 'ratio', 'country', 'date'], inplace=True, axis=1)
```

In [6]:

```
#see the new dataframe
data
```

Out[6]:

	total_vaccinations	people_vaccinated	people_fully_vaccinated	New_deaths	population
0	504502.0	448878.0	55624.0	12	40374668.0
1	547901.0	470341.0	77560.0	10	40374668.0
2	573277.0	476367.0	96910.0	10	40374668.0
3	590454.0	479372.0	111082.0	19	40374668.0
4	593313.0	479574.0	113739.0	14	40374668.0
...	...	...	...	...	...
32906	8691642.0	4814582.0	3473523.0	2	15229891.0
32907	8791728.0	4886242.0	3487962.0	2	15229891.0
32908	8845039.0	4918147.0	3493763.0	1	15229891.0
32909	8934360.0	4975433.0	3501493.0	1	15229891.0
32910	9039729.0	5053114.0	3510256.0	0	15229891.0

32911 rows × 5 columns

In [7]:

```
data.isnull().sum()
```

Out[7]:

```
total_vaccinations    0
people_vaccinated      0
people_fully_vaccinated  0
New_deaths            0
population            0
dtype: int64
```

In [8]:

```
#create a feature and target variable
features = ['total_vaccinations', 'people_vaccinated', 'people_fully_vaccinated', 'population']
x = data.loc[:, features]
y = data.loc[:, 'New_deaths']
```

In [9]:

```
#create our model and separate the data to train and test
x_train, x_test, y_train, y_test = train_test_split(x, y, random_state=0, train_size= 0.75)
```

In [10]:

```
#idea of the shape of the test and train data for x  
print("x has been split from", x.shape, "to", x_train.shape, "and", x_test.shape)
```

x has been split from (32911, 4) to (24683, 4) and (8228, 4)

In [11]:

```
#idea of the shape of the test and train data for y  
print("y has now been split from", y.shape, "to", y_train.shape, "and", y_test.shape)
```

y has now been split from (32911,) to (24683,) and (8228,)

In [12]:

```
#create the linear regression model  
linreg = LinearRegression()  
linreg.fit(x_train, y_train)
```

Out[12]:

LinearRegression()

In [13]:

```
#our prediction of new deaths  
y_pred = linreg.predict(x_test)
```

In [14]:

```
#calculate our MSE to see how bad the error is overall  
from sklearn.metrics import mean_squared_error  
error = mean_squared_error(y_test, y_pred)  
  
print(error)
```

83867.33908695345

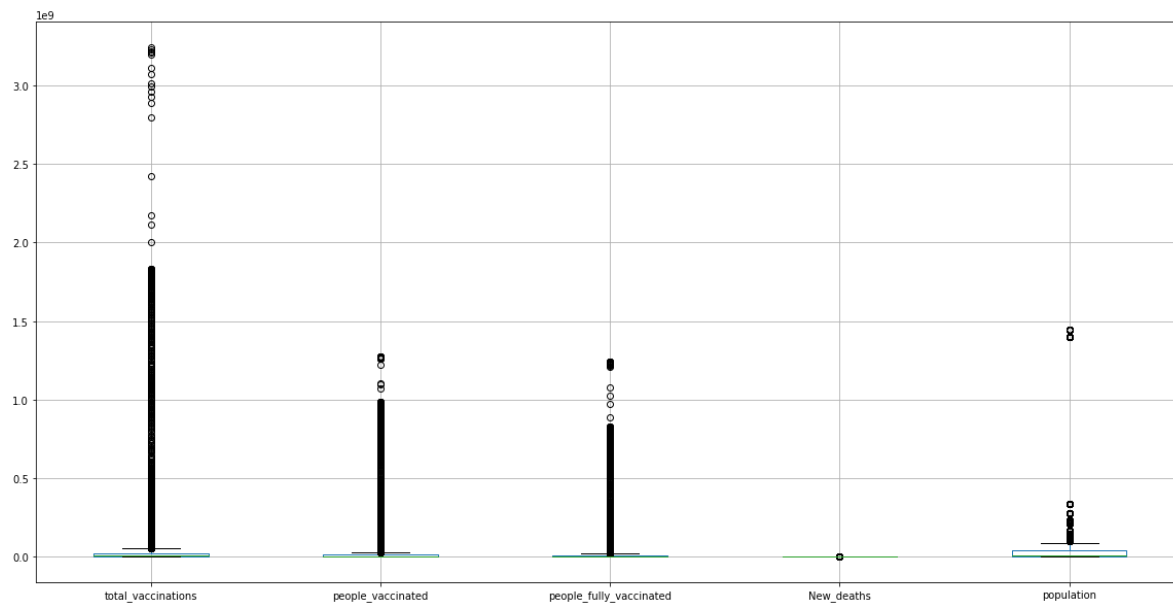
In [15]:

```
from sklearn.metrics import mean_absolute_error  
print(mean_absolute_error(y_test, y_pred))
```

111.8520793598303

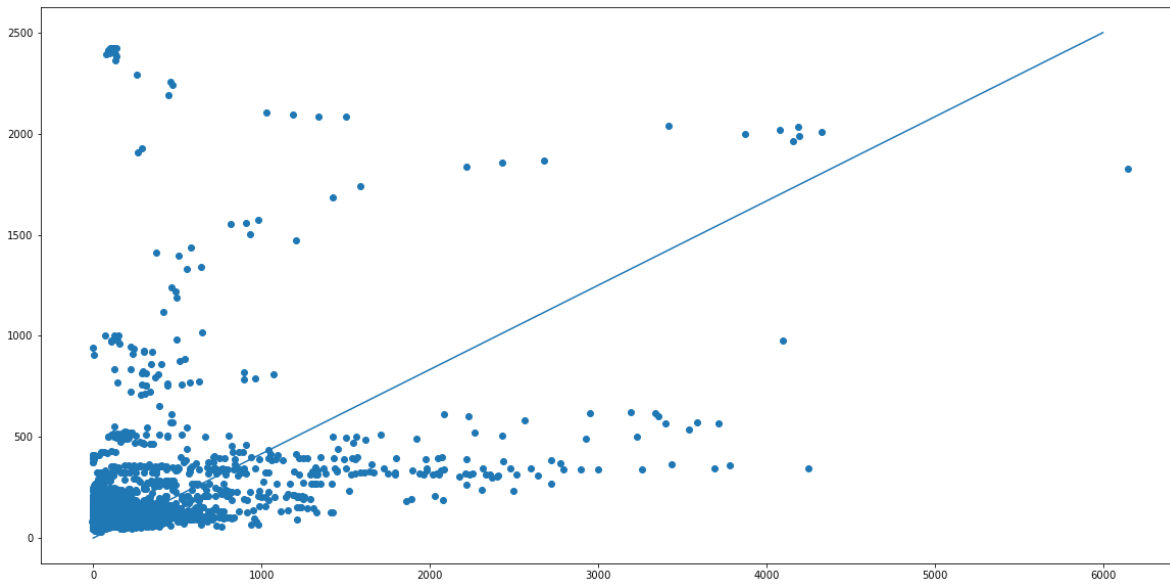
In [16]:

```
#display a box plot of the data  
plt.figure  
data.boxplot(figsize=(20,10))  
plt.show()
```



In [17]:

```
#create a scatter plot using the predicted data and a line to show it compared to the actual  
plt.figure(figsize=(20,10))  
plt.scatter(x=y_test, y=y_pred)  
plt.plot([0,6000],[0,2500])  
plt.show()
```



In []: